Mechatronics Report

Team 4 - Lea Staller (lks86), Sean Dempsey (skd82), Maddie Vercher (msv36)

Robot Design and Strategy Overview:

Mechanical Design: The base of our mechanical design was similar to the demo robot provided for the milestones. However, we added a layer of cardboard to better protect our wiring. On the robot itself, we added collector walls at the front to collect more cubes and protect the wheels from cubes, since we learned they can run over them easily. The final part of our design was a lattice-type extender wall that balanced on the robot until we moved, then detached and deployed to block the cubes from the opposing robot, allowing our robot to collect as many cubes as it wanted with no competition.

Electrical Design: We chose a simple electrical design. Given how our software worked, we did not need a color sensor. Additionally, we had the QTI sensor mounted and wired to the breadboard, but opted not to use it once we finalized our code plan. The other components wired to the breadboard were the DC motors, H-bridges, Arduino, and the battery pack.

Software Design: Our software goal was simplicity. We wanted the wall to do most of the work. Our code did not use any form of sensors, as you can see in Appendix E. We hard-coded the robot to do a lap around the board after dropping the wall, and then stop. We used time delays for the distances and turns tuned via testing different delays. An additional problem we chose to solve in software was ensuring the wall pole didn't get caught on our robot. To solve this, once we moved forward to bring the wall down, we would immediately go backward to get the pole off of ourselves. The logic behind the movements after that is shown in Appendix D in our flowchart.

Additionally, it was clear to us that to successfully deploy our wall, we needed our code to run faster than everyone else's. Our initial research determined that the main limiting factor for code running after the reset button was pushed on the Arduino is the Arduino's bootloader. Through testing, we determined that when the reset button was released, the bootloader took around 1.5 seconds before the uploaded program began running. Arduino forums[1] and official documentation[2] clearly outline how to remove the bootloader, allowing the uploaded program to run much quicker. After removing the bootloader, our code began running 0.1 seconds after releasing the reset button, which allowed the wall to deploy safely and effectively.

[1] https://arduino.stackexchange.com/a/90062

[2] https://docs.arduino.cc/built-in-examples/arduino-isp/ArduinoISP/

Design Process Reflection:

Our group decided to use a very basic robot to knock out the milestones first before progressing to the robot design we intended to use for the competition. We knew we wanted to be the first to complete the milestones, which is why we opted for this route. We did some brainstorming at first to complete milestone 1, aiming for a wedge that would drop down to push the other robot, along with enough space underneath to collect all the cubes. This idea initially had our design on the more offensive side. For milestones 2 and 3, we used the demo robot design and focused solely on the code to complete them. For milestone 4, we still used the demo design, but added a cardboard collector wall at the front to collect the cubes. Once all the milestones were completed, we moved back to our competition robot design. This is where our design changed significantly, as we began thinking outside the box. We decided to move to a more defensive stance, adding a wall to block the other robot from collecting cubes. This wall was balanced in the back of the robot, fitting in the 8” by 8” starting constraint, and when it fell, its lattice-style sides deployed and extended to be 20” on each side, as it did not need to fit inside the 12” diameter extended limit since it was not attached to the robot itself. See Appendix C for CAD images of the wall and photos of the robot in action. We kept our collector design from the original brainstorming for milestone one, with collector walls directing the cubes beneath our robot and flaps at the back to keep them there and protect the wheels. While testing this new design, we collected roughly 12 cubes beneath the robot routinely. We also kept the physical robot design pretty minimal, as we did while completing the milestones, only adding layers to protect the wiring and poles with rubber bands in the back to balance the wall.

Competition Analysis:

Our robot did not perform well at the competition. Our final stats from the round-robin were 1 win, 5 losses, and 0 ties, which ranked us 7th in Group 1. The wall mostly worked as it deployed correctly 4 out of 6 times. However, after our 4th game, one of the rubber bands broke, and we had to fix it. After that, the screws were too tight, preventing it from working as expected. Even when the wall was deployed, at times, cubes would still be on the opposing team's side, or we would get caught in the pole and not be able to grab them. An unexpected positive was that ASML really liked the uniqueness of our design. Even though it wasn’t very effective in competition, they seemed to appreciate our “out-of-the-box” approach and asked to compete against our robot after the competition was over. Our robot’s primary strengths were intimidation and uniqueness. Every time we went up to compete, the opponents appeared nervous. Additionally, no one else had a design like ours. It was unique and a new approach that no one else had decided to take. Our weaknesses were reliability and cube collection strategy, including both the physical collection system and the collection algorithm. Our wall performed consistently during testing, but when it faced another robot, many things went wrong for us. We would get caught on the pole, or the wall wouldn’t deploy, severely hampering our strategy. Additionally, our ability to successfully collect cubes was very limited. Our

code was too simple and not adaptable enough to the high variance in the wall deployment. The code worked well in testing, but with the addition of a competitor and the aforementioned issues with the wall, we were unable to adapt, as it was set to run a specific pass and then stop. We had a small frontal collection area, which significantly reduced the number of cubes that we could collect. The combination of wall deployment reliability issues and an inefficient collection strategy prevented us from scoring effectively on both sides of the game.

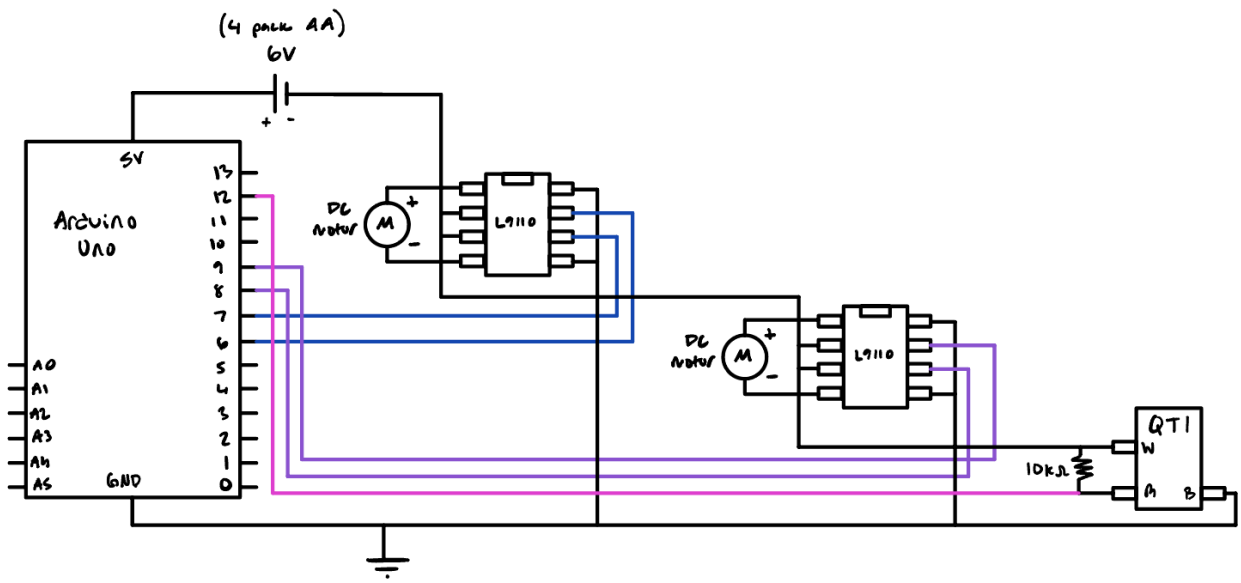
Conclusions:

Overall, our performance was poor in the competition. However, our team is proud of our creative approach. People's heads would turn whenever we did testing or stepped into the arena. If we did the project again, the main change would be our coding approach. Specifically, we would implement a more adaptive algorithm that could handle wall deployment failures and update the cube collection algorithm accordingly. We went too simple when our design was relatively complex. Ultimately, that led to our poor competition performance more than the wall not deploying properly. Additionally, ironing out the details with the wall deployment would have significantly improved our ability to deny cubes to the opposing team. Our advice to incoming students is not to be afraid to try a unique design. It may not lead to a victory, but at the end of the day, there is a lot of learning value in attempting a challenging and unconventional design. We would also recommend practicing against friends to test in a more realistic setting.

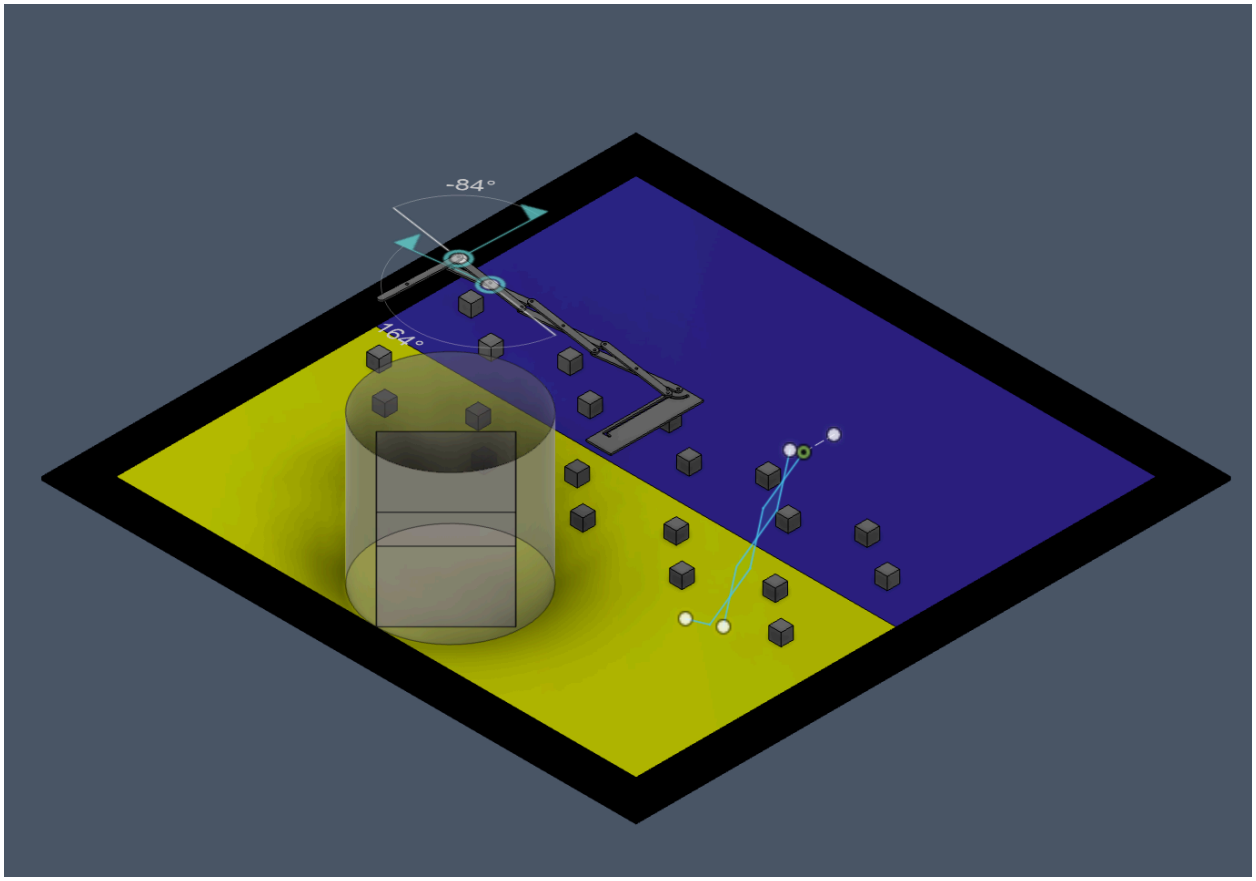
Appendix A: Bill of Materials (BOM).

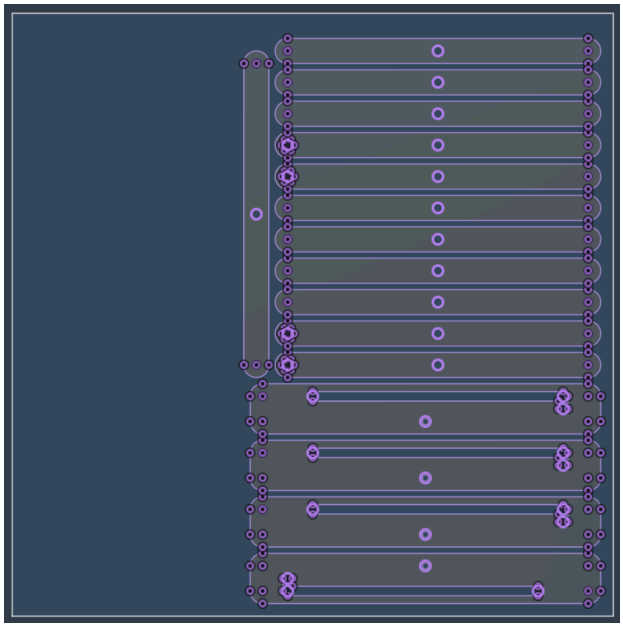
Part	Quantity	Cost (per item)
Larger wheels	2	$2.00
Acrylic	1	$5.00
Laser cutting	297.2 inches, 16 parts	$22.86
Pole for the wall	1	$4.50
Flag poles for balance	2	$0.30
Foam	1	$0.50
Hinges (not used)	2	$1.02
Total Cost:		**$39.50**

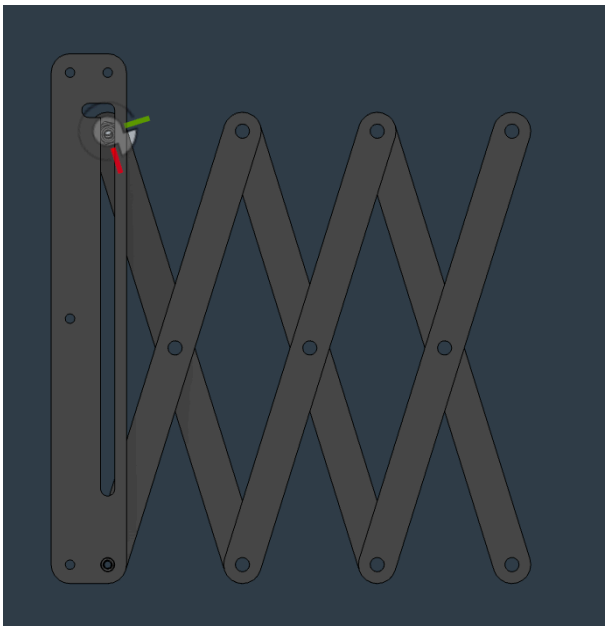
Appendix B: Circuit Diagram.

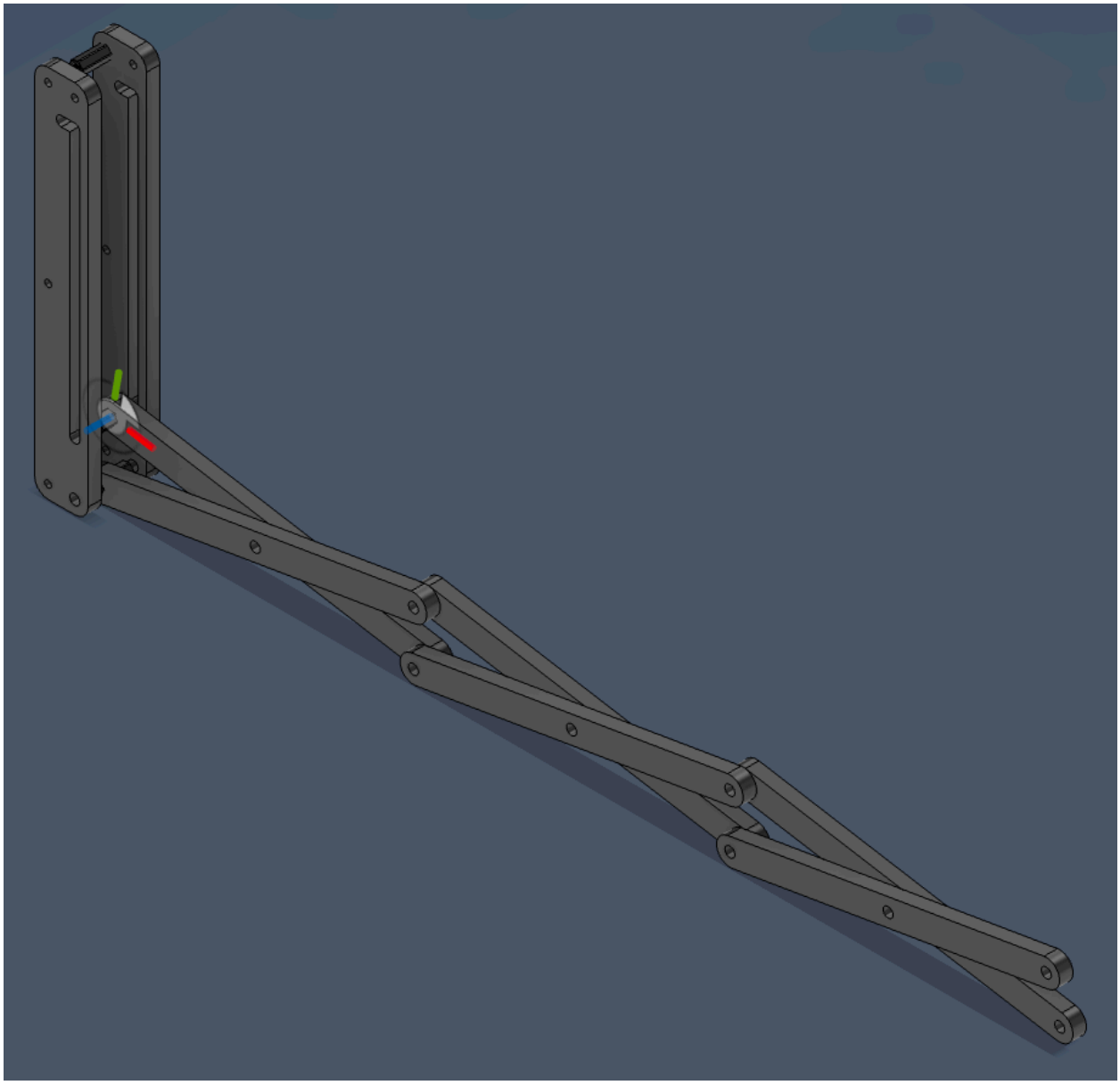


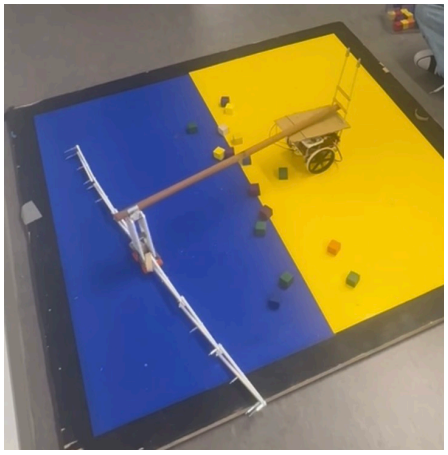
Appendix C: CAD files and drawings.

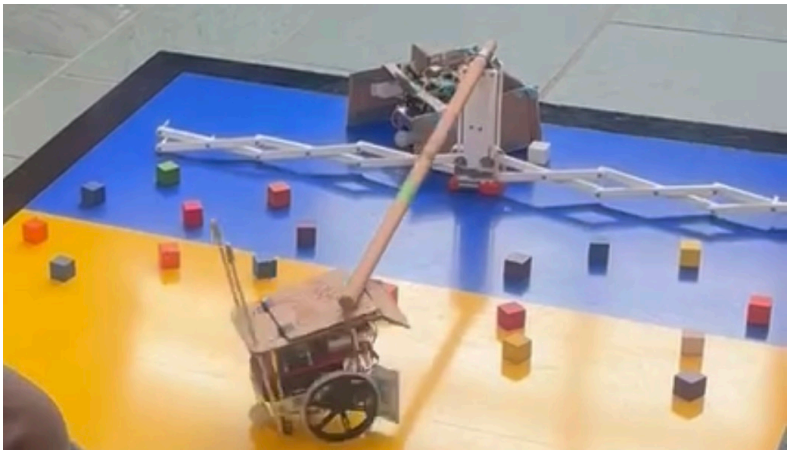


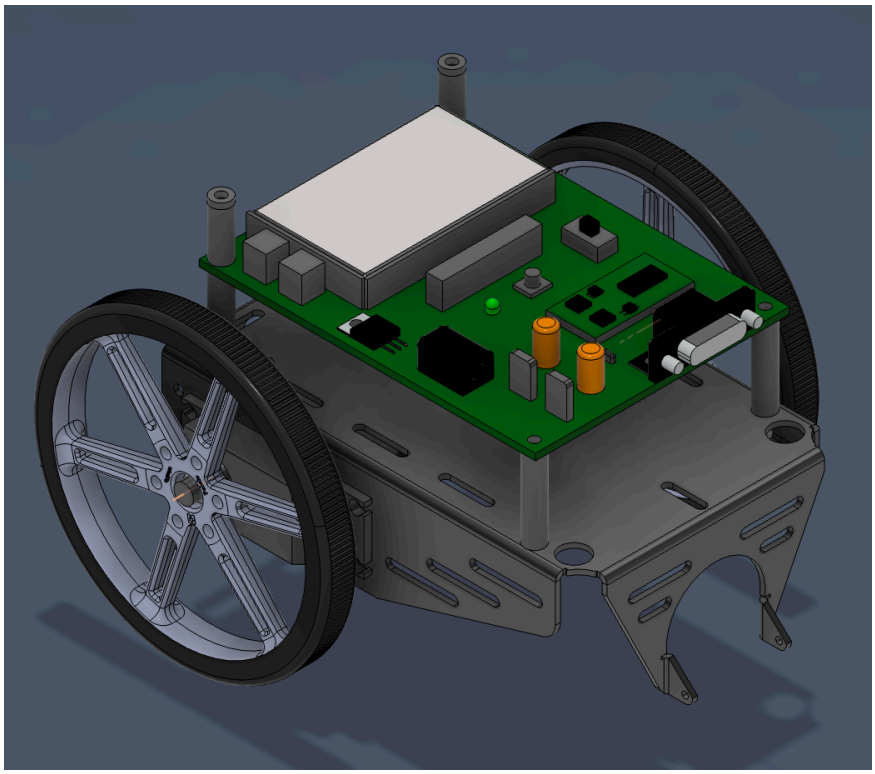






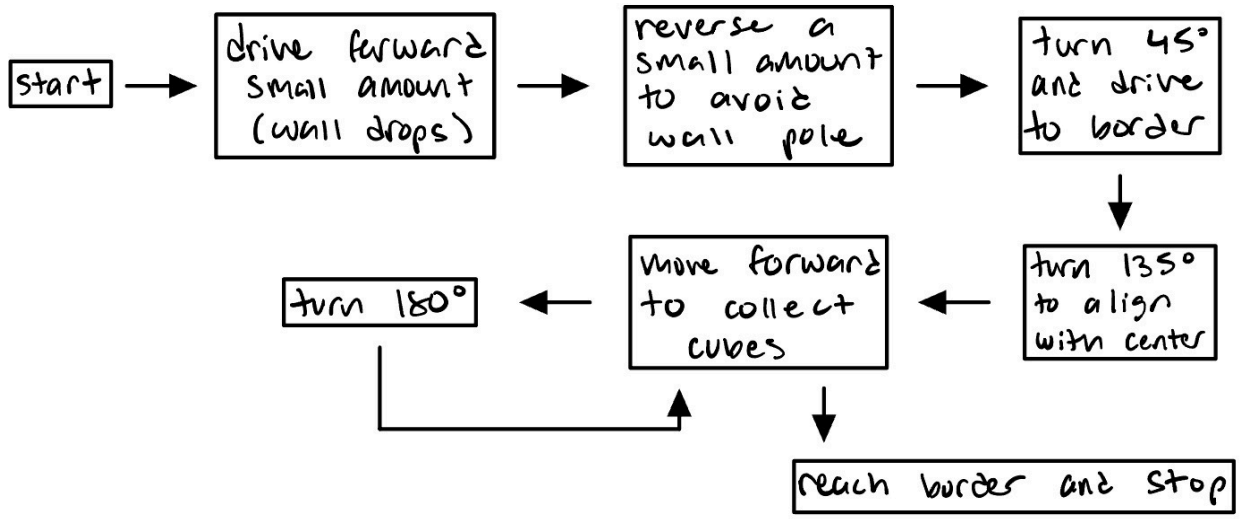






Laser Cutting	Quantity	Costs	Total
Length	297.196	$0.05	$14.86
Parts	16	$0.50	$8.00
Total			$22.86

Appendix D: Flowchart.



Appendix E: Code.

```
int main(void){ // setup code that only runs once

 // Setup
 // set pins 8 and 9 as outputs for the left motor
 DDRB = 0b00000011;
 // set pins 6 and 7 as outputs for the right motor.
 DDRD = 0b11000000;   // PD6, PD7 outputs (pins 6,7)

  while(1){ // code that loops forever
   // call functions for different directions
   drive_forward();
   _delay_ms(1000);

   drive_backward();
   _delay_ms(500);

   turn_right();
   _delay_ms(300);

   drive_forward();
   _delay_ms(2100);

   turn_left();
   _delay_ms(610);

   drive_forward();
   _delay_ms(4000);

   turn_left();
   _delay_ms(400);

   drive_forward();
   _delay_ms(200);

   turn_left();
   _delay_ms(400);
```

```
  drive_forward();
  _delay_ms(4000);

  stop();
  break; //Exit loop when done
 }
}

void drive_forward(){
 // Clear relevant bits first
 PORTB &= 0b11111100;  // clear PB0, PB1
 PORTD &= 0b00111111;  // clear PD6, PD7

 // Set forward direction
 PORTB |= 0b00000010;  // PB1 = 1, PB0 = 0
 PORTD |= 0b10000000;  // PD7 = 1, PD6 = 0
}

void drive_backward(){
 // Clear relevant bits first
 PORTB &= 0b11111100;
 PORTD &= 0b00111111;

 PORTB |= 0b00000001;  // PB0 = 1
 PORTD |= 0b01000000;  // PD6 = 1
}

void turn_left(){
 // Clear relevant bits first
 PORTB &= 0b11111100;
 PORTD &= 0b00111111;

 PORTB |= 0b00000001;  // left backward
 PORTD |= 0b10000000;  // right forward
}


void turn_right(){
 // Clear relevant bits first
 PORTB &= 0b11111100;
```

```
 PORTD &= 0b00111111;

 PORTB |= 0b00000010;  // left forward
 PORTD |= 0b01000000;  // right backward
}

void stop(){
 // Clear motor control bits
 PORTB &= 0b11111100;  // PB1 = 0, PB0 = 0
 PORTD &= 0b00111111;  // PD7 = 0, PD6 = 0
}
```